INDUSTRIAL APPLICATIONS OF MICROCONTROLLER

Problem Statement 7 | Alcohol Detection System

Project aim	Detect alcohol-related sensor response and inhibit the engine-start permit output above a calibrated threshold.
Controller	Arduino Uno (ATmega328P compatible)
Prototype mode	Wokwi-ready circuit with potentiometer input as an MQ-3 analog-output proxy; physical design uses an MQ-3 module.

Submission note: This report covers required items 1–5. The live Wokwi project is linked separately from the GitHub README for interactive circuit, code, and output review. The circuit overview in this PDF is a generated drawing, not a simulator screenshot.

1. Problem statement and aim

Develop a microcontroller-based prototype that senses alcohol-related vapour near a driver and warns the operator while inhibiting the engine-start permit output when the measured sensor signal exceeds an experimentally calibrated threshold. The prototype demonstrates a basic sensing-and-interlock sequence; it does not measure blood alcohol concentration or establish legal impairment.

Objectives

• Read the MQ-3 module analog output using Arduino Uno input A0.

• Filter readings lightly to reduce momentary variation and report values to the serial monitor.

• During sensor warm-up and above-threshold conditions, keep the active-low relay output de-energized, illuminate the red LED, and sound the buzzer.

• Below the configured threshold after warm-up, illuminate the green LED and permit the isolated relay module output.

2. Scope of the solution

The submission demonstrates the detection logic, threshold comparison, alarm outputs, and a start-permit interlock output. In Wokwi, a potentiometer provides a repeatable analog value in place of the MQ-3 module; the blue LED represents the permit output. The physical build substitutes the sensor module for the potentiometer and a properly rated isolated relay interface for the indicator.

Operating sequence

Step	Behavior
Power-up	Outputs default to inhibited; warm-up timer runs for 20 seconds.
Sampling	A0 is sampled every 100 ms and smoothed; raw and filtered values are printed.
Below threshold	Green LED on, buzzer off, active-low relay module permitted.
At/above threshold	Red LED on, buzzer sounds, relay module de-energized to inhibit start.

Limitations: Threshold 500 is illustrative and must be calibrated for the specific sensor module, power supply, placement, humidity, temperature, and clean-air baseline. A low-cost gas sensor can respond to other vapours. The system is not a certified breathalyzer or a road-ready vehicle safety device. Vehicle integration requires a qualified, isolated, fail-safe automotive design and review.

3. Required components

Category	Item	Purpose
Controller	Arduino Uno / compatible 5 V board	Analog acquisition and output control
Sensor	MQ-3 module with analog output	Alcohol-related vapour response input
Alarm	Active buzzer	Audible alert at/above threshold
Indicators	Red and green LEDs + 220 ohm resistors	Inhibit/alarm and permit status
Interlock	Active-low 5 V isolated relay module	Demonstration start-permit output
Simulation	Potentiometer in Wokwi	Controllable analog sensor proxy
Tools	Arduino IDE 2.x, USB cable, Wokwi	Compile, upload, simulate, document
Build supplies	Breadboard, jumper wires, regulated supply, enclosure	Prototype assembly

Pin connections

Uno pin	Connection
A0	MQ-3 analog output (Wokwi: potentiometer SIG)
D5	Green LED through 220 ohm resistor
D6	Red LED through 220 ohm resistor
D8	Buzzer positive; buzzer negative to GND
D9	Active-low relay module IN (Wokwi blue LED proxy)
5V / GND	Sensor/module supply and common reference; follow module ratings

Electrical caution: Do not connect a relay coil or vehicle starter directly to an Arduino GPIO. Use a relay module or transistor driver with appropriate suppression, isolation, supply, and contact rating. For an automotive system, use a protected, reviewed interface.

4. Circuit, code and output evidence

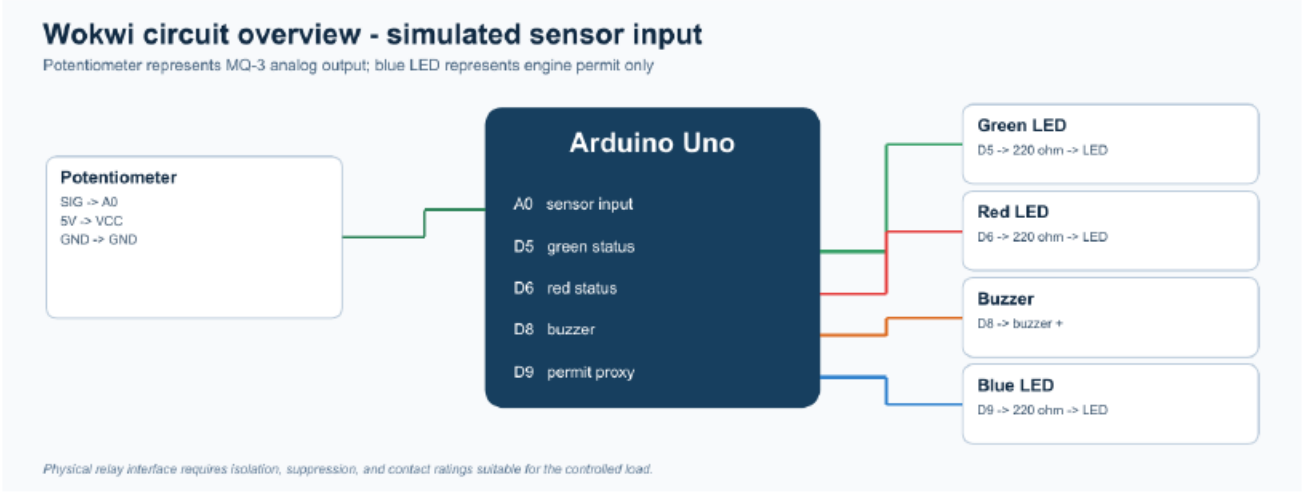
Live Wokwi project (provided separately on GitHub)

The public Wokwi simulation is linked from the repository README. It includes the circuit and sketch. Start it, wait through warm-up, then adjust the potentiometer below/above 500 to review both output states.

The circuit image below is a generated overview, not a simulator screenshot.

Expected output behavior

Input condition	Serial message / outputs
Reading below 500 after warm-up	BELOW SET THRESHOLD: start permitted | green LED and engine-proxy blue LED on | buzzer off
Reading at/above 500	ALCOHOL ABOVE SET THRESHOLD: start inhibited | red LED on | buzzer on | relay output de-energized
During 20-second warm-up	WARMUP: start inhibited | red LED on | relay output de-energized



5. Code for the solution

Complete Arduino Uno sketch. The threshold is illustrative; verify relay polarity and sensor warm-up requirements for the chosen module.

```
/*
  IAMC - Alcohol Detection and Vehicle Interlock (teaching prototype)
  Arduino Uno + MQ-3 analog output
  Outputs: buzzer D8, active-low relay driver D9, status LEDs D5/D6

  SAFETY: This is a classroom prototype, not a certified breathalyzer or
  road-ready vehicle controller. Never connect a microcontroller pin directly
  to a vehicle starter circuit. Use a properly rated isolated automotive
  interface and fail-safe review before any real deployment.
*/
const byte SENSOR_PIN = A0;
const byte GREEN_LED = 5;
const byte RED_LED = 6;
const byte BUZZER_PIN = 8;
const byte RELAY_PIN = 9; // relay module input; active LOW means engine permit
const int TRIP_THRESHOLD = 500; // illustrative ADC threshold; calibrate on hardware
const unsigned long SAMPLE_INTERVAL_MS = 100;
const unsigned long WARMUP_MS = 20000;

unsigned long startedAt;
unsigned long lastSampleAt;
int filteredReading = 0;
bool filterInitialized = false;

void setSafeState() {
  digitalWrite(RELAY_PIN, HIGH); // de-energize active-low relay: inhibit start
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);
  noTone(BUZZER_PIN);
}

void setup() {
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(RELAY_PIN, OUTPUT);
  setSafeState();
  Serial.begin(9600);
  startedAt = millis();
  Serial.println(F("IAMC alcohol detector - sensor warm-up"));
}

void loop() {
  const unsigned long now = millis();
  if (now - lastSampleAt < SAMPLE_INTERVAL_MS) return;
  lastSampleAt = now;

  const int raw = analogRead(SENSOR_PIN);
  if (!filterInitialized) {
    filteredReading = raw;
    filterInitialized = true;
  } else {
    filteredReading = (filteredReading * 3 + raw) / 4; // light smoothing
  }

  Serial.print(F("raw="));
  Serial.print(raw);
  Serial.print(F(" filtered="));
  Serial.println(filteredReading);

  if (now - startedAt < WARMUP_MS) {
    setSafeState();
    Serial.println(F("WARMUP: start inhibited"));
    return;
  }

  if (filteredReading >= TRIP_THRESHOLD) {
    setSafeState();
    tone(BUZZER_PIN, 2200);
    Serial.println(F("ALCOHOL ABOVE SET THRESHOLD: start inhibited"));
  } else {
    digitalWrite(RELAY_PIN, LOW); // energize active-low relay module: permit start
    digitalWrite(GREEN_LED, HIGH);
    digitalWrite(RED_LED, LOW);
    noTone(BUZZER_PIN);
    Serial.println(F("BELOW SET THRESHOLD: start permitted"));
  }
}
```

Submission checklist

The PDF covers required items 1–5. The interactive Wokwi project is provided separately and linked from the GitHub README.
It exposes the circuit, sketch, and permit/inhibit outputs after warm-up.
The generated circuit overview on page 4 is not a simulator screenshot.

The demo video and Gerber package are also available in the repository.

Project files

README.md | sketch.ino | diagram.json | wokwi.toml | circuit-overview.pdf | IAMC_Alcohol_Detection_Demo.mp4 | Gerber_Files/

End of report